Assignment 2.1

Name: Sridhar Surapaneni Date:15th Sep 2024

For this assignment, you will refer to the textbook to solve the practice exercises. Use Python to answer any coding problems (not R, even if indicated in your textbook). Use Jupyter Notebook, Google Colab, or a similar software program to complete your assignment. Submit your answers as a PDF or HTML file. As a best practice, always label your axes and provide titles for any graphs generated on this assignment. Round all quantitative answers to 2 decimal places.

Problem # 2.1.

For the rain simulation example in Section 2.1.1, but with probability of rain 0.30 on any given day, simulate the outcome (a) on the next day, (b) the next 10 days. (c) Simulate the proportion of days of rain for the next (i) 100 days, (ii) 10,000 days, (iii) 1,000,000 days. Use the simulation to explain the long-run relative frequency definition of probability.

`(a) Your answer goes here`

In [2]:

```
import random

probability_of_rain = 0.30
if random.random() < probability_of_rain:
  outcome = "Rain"
else:
  outcome = "No Rain"
print("Outcome for the next day:", outcome)
```

```
Outcome for the next day: Rain
Outcomes for the next 10 days: ['Rain', 'No Rain', 'Rain', 'Rain', 'Rain', 'No Rain', 'No
Rain', 'No Rain', 'No Rain', 'No Rain']
Proportion of rain days for the next 100 days: 0.19
Proportion of rain days for the next 10,000 days: 0.3025
Proportion of rain days for the next 1,000,000 days: 0.300766
```

`(b) Your answer goes here`

In [3]:

```
outcomes_10_days = []
for _ in range(10):
  if random.random() < probability_of_rain:
    outcomes_10_days.append("Rain")
  else:
    outcomes_10_days.append("No Rain")
print("Outcomes for the next 10 days:", outcomes_10_days)
```

```
Outcomes for the next 10 days: ['No Rain', 'Rain', 'No Rain', 'No Rain', 'Rain', 'Rain',
'No Rain', 'Rain', 'Rain', 'No Rain']
```

`(c) Your answer goes here`

In [6]:

```
def simulate_rain_proportion(num_days):
  rain_days = 0
  for _ in range(num_days):
    if random.random() < probability_of_rain:
```

```
        rain_days += 1
    return rain_days / num_days

# (i) 100 days
proportion_100_days = simulate_rain_proportion(100)
print("Proportion of rain days for the next 100 days:", round(proportion_100_days,2))

# (ii) 10,000 days
proportion_10000_days = simulate_rain_proportion(10000)
print("Proportion of rain days for the next 10,000 days:", round(proportion_10000_days,2)
)

# (iii) 1,000,000 days
proportion_1000000_days = simulate_rain_proportion(1000000)
print("Proportion of rain days for the next 1,000,000 days:", round(proportion_1000000_da
ys,2))
```

```
Proportion of rain days for the next 100 days: 0.3
Proportion of rain days for the next 10,000 days: 0.3
Proportion of rain days for the next 1,000,000 days: 0.3
```

Problem # 2.2.

Data analysts often implement statistical inference methods by setting the probability of a correct inference equal to 0.95. Let *A* denote the event that an inference for the population about men is correct. Let *B* represent the event of a corresponding inference about women being correct. Suppose that these are independent events.

(a) Find the probability that (i) *both* inferences are correct, (ii) *neither* inference is correct.
(b) Construct the probability distribution for *Y* = number of correct inferences.
(c) With what probability would each inference need to be correct in order for the probability to be 0.95 that *both* are correct?

```
(a) Your answer goes here
```

In [7]:

```
probability_A = 0.95
probability_B = 0.95
probability_both_correct = probability_A * probability_B
print("Probability that both inferences are correct:", round(probability_both_correct,2))

#(ii) Probability that neither inference is correct
probability_A_incorrect = 1 - probability_A
probability_B_incorrect = 1 - probability_B
probability_neither_correct = probability_A_incorrect * probability_B_incorrect
print("Probability that neither inference is correct:", round(probability_neither_correct
,2))
```

```
Probability that both inferences are correct: 0.9
Probability that neither inference is correct: 0.0
```

```
(b) Your answer goes here
```

In [8]:

```
# Probability distribution for Y = number of correct inferences
# Y can take values 0, 1, or 2

probability_Y_0 = probability_neither_correct  # Probability that neither is correct
probability_Y_2 = probability_both_correct  # Probability that both are correct

# Probability that exactly one is correct
probability_Y_1 = (probability_A * (1 - probability_B)) + ((1 - probability_A) * probabi
lity_B)

print("Probability Distribution for Y:")
```

```
print("Y = 0:", round(probability_Y_0, 2))
print("Y = 1:", round(probability_Y_1, 2))
print("Y = 2:", round(probability_Y_2, 2))
```

```
Probability Distribution for Y:
Y = 0: 0.0
Y = 1: 0.1
Y = 2: 0.9
```

`(c) Your answer goes here`

In [9]:

```
# We want P(A and B) = 0.95
# P(A and B) = P(A) * P(B) (since A and B are independent)
# We want P(A) * P(B) = 0.95
# Let P(A) = P(B) = p
# Then p * p = 0.95
# p^2 = 0.95
# p = sqrt(0.95)

p = (0.95)**0.5
print("Probability that each inference needs to be correct:", round(p, 2))
```

```
Probability that each inference needs to be correct: 0.97
```

Problem # 2.4.

A wine connoisseur is asked to match five glasses of red wine with the bottles from which they came, representing five different grape types.

(a) Set up a sample space for the five guesses.
(b) With random guessing, find the probability of getting all five correct.

`(a) Your answer goes here`

The sample space consists of all possible permutations of the five glasses, sample_space = list(itertools.permutations([1, 2, 3, 4, 5])), the length of sample space is 120. Example: (3, 2, 4, 1, 5), (3, 2, 4, 5, 1), (3, 2, 5, 1, 4), (3, 2, 5, 4, 1)

`(b) Your answer goes here`

In [13]:

```
import itertools

sample_space = list(itertools.permutations([1, 2, 3, 4, 5]))
print("Sample space size:", len(sample_space))

# (b) With random guessing, find the probability of getting all five correct.
# The probability of getting all five correct with random guessing is 1 / number of permu
tations.
probability_all_correct = 1 / len(sample_space)
print("Probability of getting all five correct with random guessing:", round(probability_
all_correct, 2))
```

```
Sample space size: 120
Probability of getting all five correct with random guessing: 0.01
```

Problem # 2.15.

Each week an insurance company records Y = number of payments because of a home burning down. State conditions under which we would expect Y to approximately have a Poisson distribution.

```
Your answer goes here
```

We would expect Y to approximately have a Poisson distribution under the following conditions:

1. **The events (home burnings) occur independently of each other.**
2. **The probability of a home burning down in a short time interval is proportional to the length of the interval.**
3. **The probability of more than one home burning down in a very short time interval is negligible.**
4. **The rate of home burnings is constant over time. In simpler terms, the events should be rare, random, and independent, and the average rate of events should be relatively constant.**

Problem # 2.16.

Each day a hospital records the number of people who come to the emergency room for treatment.

(a) In the first week, the observations from Sunday to Saturday are 10, 8, 14, 7, 21, 44, 60. Do you think that the Poisson distribution might describe the random variability of this phenomenon adequately. Why or why not?
(b) Would you expect the Poisson distribution to better describe, or more poorly describe, the number of weekly admissions to the hospital for a rare disease? Why?

```
(a) Your answer goes here
```

In [14]:

```
observations = [10, 8, 14, 7, 21, 44, 60]
```

```
Based on the observations, the Poisson distribution might not be a good fit for describin
g the random variability of emergency room visits.
The observations show a large variation, with some days having significantly more visits
than others.
This suggests that the average rate of emergency room visits is not constant throughout t
he week, which is a key assumption of the Poisson distribution.
```

```
(b) Your answer goes here
```

In [15]:

```

```

```
For a rare disease, the Poisson distribution would likely be a better fit.
This is because the events (admissions for a rare disease) are likely to be independent,
and the average rate of admissions would be relatively constant over time.
The Poisson distribution assumes a constant average rate of events over time.
In the case of a rare disease, the number of weekly admissions would likely be relatively
small and stable, with occasional fluctuations due to random chance.
This aligns better with the assumptions of the Poisson distribution compared to the
emergency room visits, where external factors and varying rates might influence the data.
```

Problem # 2.17.

An instructor gives a course grade of B to students who have total score on exams and homeworks between 800 and 900, where the maximum possible is 1000. If the total scores have approximately a normal distribution with mean 830 and standard deviation 50, about what proportion of the students receive a B?

```
Your answer goes here
```

In [16]:

```
from scipy.stats import norm
```

```
mean = 830
std_dev = 50

# Calculate the z-scores for 800 and 900
z_score_800 = (800 - mean) / std_dev
z_score_900 = (900 - mean) / std_dev

# Calculate the probability of a score between 800 and 900 using the cumulative distribut
ion function (CDF)
proportion_B = norm.cdf(z_score_900) - norm.cdf(z_score_800)

print("Proportion of students receiving a B:", round(proportion_B, 2))
```

```
Proportion of students receiving a B: 0.64
```

Problem # 2.20.

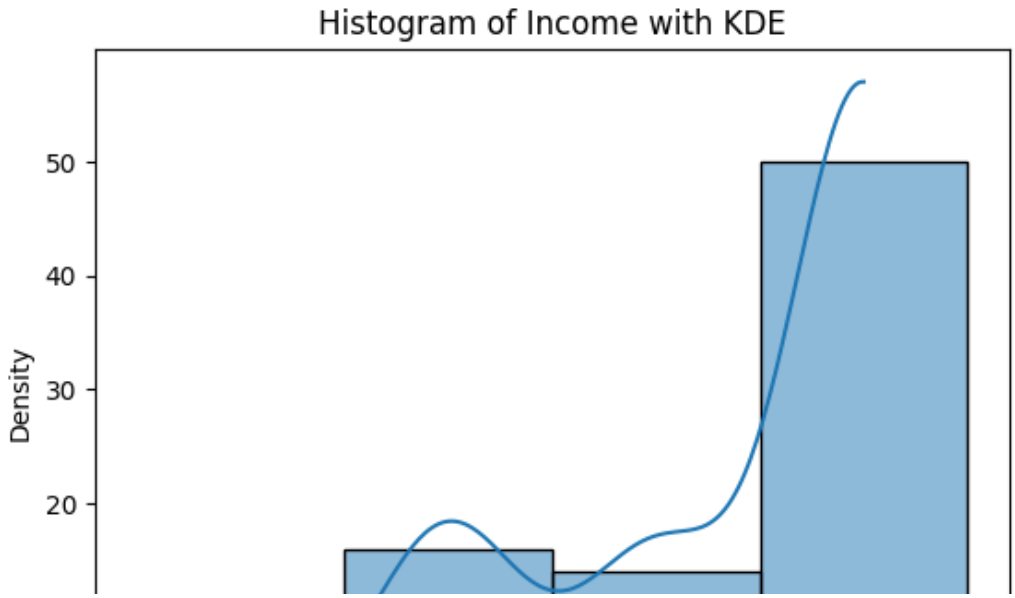
Create a data file with the income values in the `Income` data file at the text website.

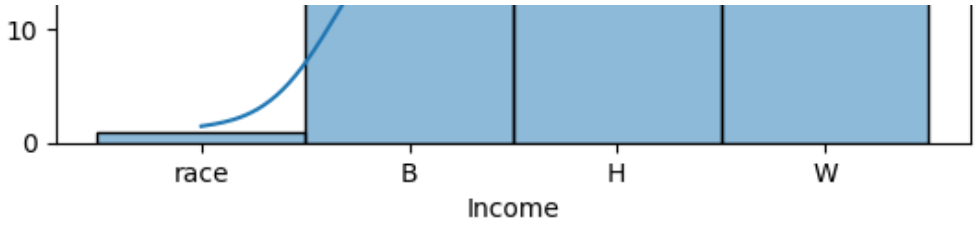
(a) Construct a histogram or a smooth-curve approximation for the *pdf* of income in the corresponding population by plotting results using the density function in R (explained in Exercise 1.18).
(b) Of the probability distributions studied in this chapter, which do you think might be most appropriate for these data? Why? Plot the probability function of that distribution having the same mean and standard deviation as the income values. Does it seem to describe the income distribution well?

`(a) Your answer goes here`

In [29]:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Read the data from the income.dat file
try:
    income_data = pd.read_csv('Income.dat', header=None, names=['Income'], delim_whitesp
ace=True) # Added delim_whitespace=True to handle potential whitespace as delimiter
except FileNotFoundError:
    print("Income.dat file not found. Please upload the file to your Colab environment.")

# (a) Construct a histogram or a smooth-curve approximation for the pdf of income
if 'income_data' in locals():
    sns.histplot(income_data['Income'], kde=True)
    plt.title('Histogram of Income with KDE')
    plt.xlabel('Income')
    plt.ylabel('Density')
    plt.show()
```





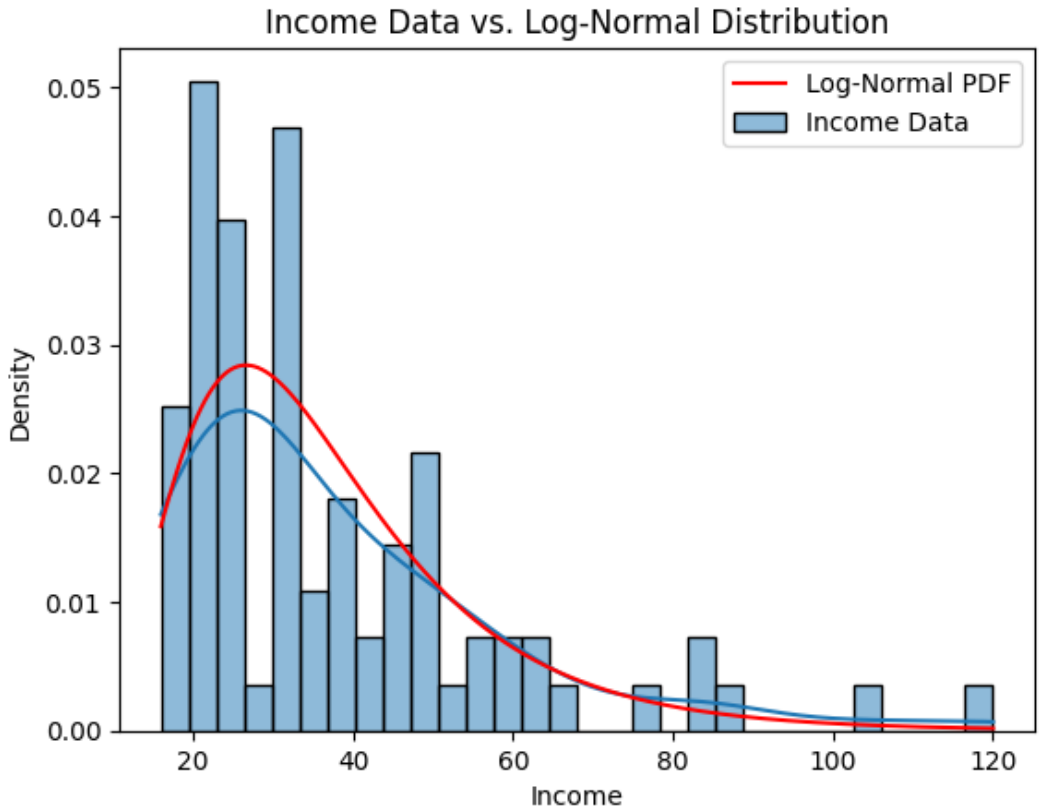
(b) Your answer goes here

In [47]:

```
import numpy as np
from scipy.stats import lognorm
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
try:
    # Specify column names based on the structure of the file
    income_data = pd.read_csv('Income.dat', delim_whitespace=True, names=['Income', 'Edu
cation', 'Race'], skiprows=1)
except FileNotFoundError:
    print("Income.dat file not found. Please upload the file to your environment.")

# Assuming income_data is already loaded as a DataFrame
if 'income_data' in locals():
    # Inspect the first few rows of the dataset
    print("First few rows of income_data:")
    print(income_data.head(10))

    # Extract the 'Income' column for analysis
    income_column = income_data['Income']

    # Convert 'Income' to numeric, forcing errors to NaN, then drop NaN values
    income_column = pd.to_numeric(income_column, errors='coerce')

    # Remove NaN, infinite, and non-positive values
    income_data_clean = income_column[np.isfinite(income_column) & (income_column > 0)]

    # Diagnostic: Check the number of data points
    print(f"Number of valid income data points: {len(income_data_clean)}")
    print(f"Income data statistics:\n{income_data_clean.describe()}")

    # Proceed if there are enough data points
    if len(income_data_clean) > 0:
        # Calculate the log of income data
        log_income = np.log(income_data_clean)

        # Calculate mean and std of the log-transformed income
        mu = np.mean(log_income)
        sigma = np.std(log_income)

        # Diagnostic: Print mean and std
        print(f"Log-Income mean: {mu}, Log-Income std: {sigma}")

        # Generate the log-normal distribution with the same mean and std
        x = np.linspace(income_data_clean.min(), income_data_clean.max(), 1000)
        pdf = lognorm.pdf(x, s=sigma, scale=np.exp(mu))

        # Plot the histogram and the log-normal PDF
        sns.histplot(income_data_clean, kde=True, stat='density', bins=30, label='Income
Data')
        plt.plot(x, pdf, 'r-', label='Log-Normal PDF')
        plt.title('Income Data vs. Log-Normal Distribution')
        plt.xlabel('Income')
        plt.ylabel('Density')
        plt.legend()

        # Show the plot
        plt.show()
```

```
    else:
        print("No valid data points to plot.")
```

```
First few rows of income_data:
   Income  Education Race
0      16         10    B
1      18          7    B
2      26          9    B
3      16         11    B
4      34         14    B
5      22         12    B
6      42         16    B
7      42         16    B
8      16          9    B
9      20         10    B
Number of valid income data points: 80
Income data statistics:
count     80.000000
mean      37.525000
std       20.672843
min       16.000000
25%       22.000000
50%       30.000000
75%       46.500000
max      120.000000
Name: Income, dtype: float64
Log-Income mean: 3.5047200467142465, Log-Income std: 0.4721297538269946
```



Problem # 2.21.

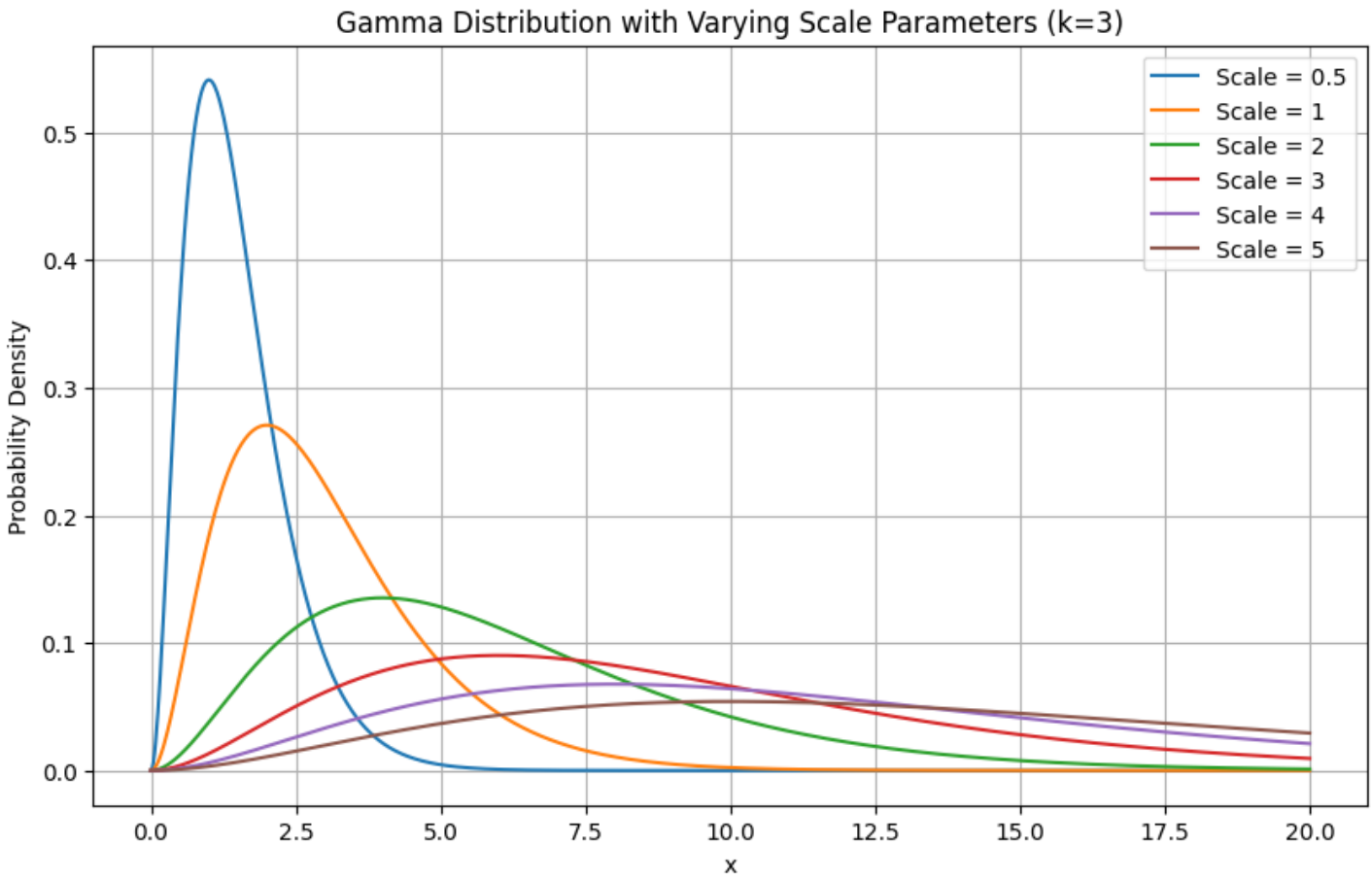
Plot the gamma distribution by fixing the shape parameter $k = 3$ and setting the scale parameter = 0.5, 1, 2, 3, 4, 5. What is the effect of increasing the scale parameter? (See also Exercise 2.48.)

```
Your answer goes here
```

In [48]:

```
from scipy.stats import gamma
import matplotlib.pyplot as plt
import numpy as np
```

```
k = 3  # Shape parameter
scale_params = [0.5, 1, 2, 3, 4, 5]

x = np.linspace(0, 20, 1000)

plt.figure(figsize=(10, 6))

for scale in scale_params:
    pdf = gamma.pdf(x, a=k, scale=scale)
    plt.plot(x, pdf, label=f'Scale = {scale}')

plt.title('Gamma Distribution with Varying Scale Parameters (k=3)')
plt.xlabel('x')
plt.ylabel('Probability Density')
plt.legend()
plt.grid(True)
plt.show()
```



Problem # 2.22.

Consider the mammogram diagnostic example in Section 2.1.4.

(a) Show that the joint probability distribution of diagnosis and disease status is as shown in Table 2.6. Given that a diagnostic test result is positive, explain how this joint distribution shows that the 12% of incorrect diagnoses for the 99% of women not having breast cancer swamp the 86% of correct diagnoses for the 1% of women actually having breast cancer.
(b) The first test for detecting HIV-positive status had a sensitivity of 0.999 and specificity of 0.9999. Explain what these mean. If at that time 1 in 10,000 men were truly HIVpositive, find the positive predictive value. Based on this example, explain the potential disadvantage of routine diagnostic screening of a population for a rare disease.

TABLE 2.6 Joint probability distribution for disease status and diagnosis of breast cancer mammogram, based on conditional probabilities in Table 2.1

Disease Status	Diagnosis from Mammogram

Disease Status	Diagnosis from Mammogram	Positive (+)	Negative (-)	Total
Yes (D)		0.0086	0.0014	0.01
No (D^c)		0.1188	0.8712	0.99

```
Your answer goes here
```

In [49]:

```
# P(D) = 0.01 (Probability of having breast cancer)
# P(D^c) = 0.99 (Probability of not having breast cancer)
# P(+|D) = 0.86 (Sensitivity: Probability of a positive test given disease)
# P(-|D) = 0.14 (Probability of a negative test given disease)
# P(+|D^c) = 0.12 (False positive rate: Probability of a positive test given no disease)
# P(-|D^c) = 0.88 (Probability of a negative test given no disease)

# Calculate the joint probabilities
P_plus_and_D = P_D = 0.01 * 0.86  # P(+ and D) = P(D) * P(+|D)
P_minus_and_D = P_D * 0.14  # P(- and D) = P(D) * P(-|D)
P_plus_and_not_D = P_not_D = 0.99 * 0.12  # P(+ and D^c) = P(D^c) * P(+|D^c)
P_minus_and_not_D = P_not_D * 0.88  # P(- and D^c) = P(D^c) * P(-|D^c)

print("Joint Probability Table:")
print("| Disease Status | Diagnosis from Mammogram | Positive (+) | Negative (-) | Total
|")
print("|--------------------|------------------------------|--------------|-----------|
-------|")
print(f"| Yes (D)            | {round(P_plus_and_D, 4)}                        | {round(P
_minus_and_D, 4)}        | {round(P_plus_and_D + P_minus_and_D, 4)}      |")
print(f"| No (D^c)          | {round(P_plus_and_not_D, 4)}                        | {round
(P_minus_and_not_D, 4)}        | {round(P_plus_and_not_D + P_minus_and_not_D, 4)}      |")

# Explanation:
# P(+|D) is 0.86 which is the probability of a correct diagnosis for the 1% of women actu
ally having breast cancer.
# P(+|D^c) is 0.12 which is the probability of an incorrect diagnosis for the 99% of wome
n not having breast cancer.
# The false positive rate (0.12) is much higher than the true positive rate (0.86).
# The number of false positives (0.99 * 0.12 = 0.1188) swamps the number of true positive
s (0.01 * 0.86 = 0.0086).
# Therefore, when the test result is positive, it is more likely that the woman does not
have breast cancer, even though the test is positive.

# Calculate P(D|+)
P_plus = P_plus_and_D + P_plus_and_not_D # P(+)
P_D_given_plus = P_plus_and_D / P_plus

print(f"P(D|+): {round(P_D_given_plus, 4)}")
```

```
Joint Probability Table:
| Disease Status | Diagnosis from Mammogram | Positive (+) | Negative (-) | Total |
|--------------------|------------------------------|--------------|-----------|-------
|
| Yes (D)            | 0.0086                       | 0.0012       | 0.0098     |
| No (D^c)          | 0.1188                       | 0.1045       | 0.2233     |
P(D|+): 0.0675
```

Problem # 2.27.

The distribution of *X* = heights *(cm)* of women in the U.K. is approximately *N*(162, 7^2). Conditional on *X* = *x*, suppose *Y* = weight *(kg)* has a *N*(3.0 + 0.40x, 8^2) distribution. Simulate and plot 1000 observations from this approximate bivariate normal distribution. Approximate the marginal means and standard deviations for *X* and *Y*. Approximate and interpret the correlation.

```
Your answer goes here
```

In [51]:

In [31]:

```
# Sensitivity: The probability of a positive test result given that the person truly has
the disease.
# In this case, it's 0.999, meaning that if a person has HIV, there's a 99.9% chance the
test will correctly identify them as positive.

# Specificity: The probability of a negative test result given that the person does not h
ave the disease.
# In this case, it's 0.9999, meaning that if a person does not have HIV, there's a 99.99%
chance the test will correctly identify them as negative.

# Prevalence: The proportion of the population that truly has the disease.
# In this case, it's 1 in 10,000, or 0.0001.

# Positive Predictive Value (PPV): The probability that a person truly has the disease gi
ven that their test result is positive.
# We can calculate PPV using Bayes' theorem:
# PPV = (Sensitivity * Prevalence) / [(Sensitivity * Prevalence) + ((1 - Specificity) *
(1 - Prevalence))]

sensitivity = 0.999
specificity = 0.9999
prevalence = 0.0001

ppv = (sensitivity * prevalence) / ((sensitivity * prevalence) + ((1 - specificity) * (1
- prevalence)))

print("Positive Predictive Value (PPV):", round(ppv, 2))
```

```
Positive Predictive Value (PPV): 0.5
```